



BAYU LIBRARY

READ · LEARN · GROW

Panduan Persembahan ERD (ERD Presentation Script) — Skrip langkah demi langkah untuk menerangkan Entity Relationship Diagram sistem ini

Bayu Library Management System · ICT502 Database Engineering · Group 2, NBSC2306A

1. Struktur Keseluruhan	Overview
2. PERSON — Supertype	Step 2
3. MEMBER & LIBRARIAN — Subtype (1:1)	Step 3
4. CATEGORY — Self-Referencing	Step 4
5. AUTHOR, BOOK & BOOK_AUTHOR — M:N	Step 5
6. BORROWING — Associative Entity	Step 6
7. Business Rules → Constraints	Step 7
8. PL/SQL — Function, Trigger & View	Step 8
9. Soalan Lazim Penilai (Q&A)	Step 9
10. Tips Penyampaian	Step 10

1. Struktur Keseluruhan — Mula Presentation Anda

Cara membuka presentation supaya penilai nampak "big picture" dahulu sebelum masuk detail.

"Sistem ni ada **8 entiti** yang saya kelompokkan kepada tiga kategori logik: pertama **Orang (Person)** — Person, Member, Librarian; kedua **Katalog** — Category, Author, Book; dan ketiga **Transaksi** — Borrowing, yang menghubungkan kesemuanya."

PERSON

— **ISA** → Supertype yang dikongsi oleh Member & Librarian (Langkah 2 & 3)

CATEGORY

— **1:M** → BOOK, yang juga M:N dengan AUTHOR melalui BOOK_AUTHOR (Langkah 4 & 5)

BORROWING

— **3 FK** → Menghubungkan MEMBER, BOOK, dan LIBRARIAN dalam satu transaksi (Langkah 6)

Kenapa urutan ini penting: Jangan baca jadual ikut abjad. Bermula dari entiti "pusat" (Person) dan bergerak keluar mengikut logik perhubungan buat presentation jauh lebih mudah diikuti — dan menunjukkan anda faham *kenapa* schema direka begini, bukan sekadar menghafal nama jadual.

2. Entiti PERSON — Supertype

Entiti asas yang dikongsi oleh Member dan Librarian.

PERSON	
PERSON_ID	VARCHAR2(10) — Primary Key
FULL_NAME	VARCHAR2(100) NOT NULL
EMAIL	VARCHAR2(100) NOT NULL, UNIQUE
PHONE	VARCHAR2(20)
ADDRESS	VARCHAR2(255)
GENDER	CHAR(1) — CHECK IN ('M','F')
PERSON_TYPE	VARCHAR2(10) — CHECK IN ('MEMBER','LIBRARIAN')

"Member dan Librarian kedua-duanya ada nama, email, dan telefon. Supaya attribute yang sama ini tidak disimpan dua kali (redundant), saya **extract** jadi satu supertype entity bernama `PERSON`. Ini dipanggil **generalization / specialization relationship** — dalam ERD biasa ditunjukkan dengan simbol ISA (segitiga)."

PERSON_TYPE berfungsi sebagai *discriminator* — cara pantas untuk tahu role seseorang tanpa perlu JOIN ke jadual MEMBER atau LIBRARIAN.

3. MEMBER & LIBRARIAN — Subtype (Relationship 1:1)

Bagaimana PERSON "berpisah" jadi dua role yang berbeza.

MEMBER	
MEMBER_ID	VARCHAR2(10) — Primary Key
PERSON_ID FK → PERSON	VARCHAR2(10) NOT NULL, UNIQUE
MEMBERSHIP_DATE	DATE NOT NULL
MEMBERSHIP_TYPE	VARCHAR2(20) — CHECK IN ('STANDARD','PREMIUM')
STATUS	VARCHAR2(10) — CHECK IN ('ACTIVE','INACTIVE')
PASSWORD_HASH	VARCHAR2(100)

LIBRARIAN	
LIBRARIAN_ID	VARCHAR2(10) — Primary Key
PERSON_ID FK → PERSON	VARCHAR2(10) NOT NULL, UNIQUE
STAFF_ID	VARCHAR2(20) NOT NULL, UNIQUE
POSITION	VARCHAR2(50) NOT NULL
PASSWORD_HASH	VARCHAR2(100)

"Setiap PERSON cuma boleh jadi **satu** — Member ATAU Librarian, tak boleh dua-dua sekali. Saya enforce peraturan ini dengan letak **UNIQUE constraint** atas column PERSON_ID dalam kedua-dua jadual anak. Relationship-nya **1:1 mandatory** dari sudut Member/Librarian (setiap rekod mesti ada Person), tapi **optional** dari sudut Person (boleh wujud Person tanpa jadi Member/Librarian lagi)."

Soalan dijangka: "Kenapa tak gabung je semua dalam satu jadual dengan column PERSON_TYPE?"

"Sebab Librarian ada attribute (STAFF_ID, POSITION) yang tak relevan untuk Member, dan Member ada attribute (MEMBERSHIP_TYPE) yang tak relevan untuk Librarian. Kalau digabungkan, akan banyak column bernilai NULL — ini melanggar prinsip normalization (setiap jadual hanya simpan attribute yang relevan kepada entiti tersebut)."

4. CATEGORY — Self-Referencing Relationship

Bagaimana satu jadual boleh berhubung dengan dirinya sendiri.

CATEGORY	
CATEGORY_ID	VARCHAR2(10) — Primary Key
CATEGORY_NAME	VARCHAR2(100) NOT NULL
PARENT_ID FK → CATEGORY	VARCHAR2(10), nullable

"Satu kategori boleh ada sub-kategori — contoh 'Science' sebagai induk (parent), 'Physics' dan 'Biology' sebagai anak. Ini dipanggil **unary / self-referencing relationship**, dengan cardinality **1:M** — satu kategori boleh ada banyak anak, tetapi setiap anak cuma ada satu induk (atau tiada langsung, jika dia top-level)."

Tip visual: Dalam ERD, relationship self-referencing biasanya digambar sebagai gelung (loop) yang keluar dan masuk semula ke kotak entiti yang sama. Sebut perkataan "recursive relationship" — ini istilah teknikal yang penilai akan cari.

5. AUTHOR, BOOK & BOOK_AUTHOR — Menyelesaikan Many-to-Many

Bahagian paling penting untuk tunjukkan pemahaman normalization.

BOOK	
BOOK_ID	VARCHAR2(10) — Primary Key
CATEGORY_ID FK → CATEGORY	VARCHAR2(10) NOT NULL
TITLE	VARCHAR2(255) NOT NULL
ISBN	VARCHAR2(20) NOT NULL, UNIQUE
PUBLISH_YEAR	NUMBER — CHECK BETWEEN 1000 AND 9999
PUBLISHER	VARCHAR2(100)
TOTAL_COPIES	NUMBER — CHECK ≥ 0
AVAILABLE_COPIES	NUMBER — CHECK ≥ 0 AND ≤ TOTAL_COPIES

AUTHOR	
AUTHOR_ID	VARCHAR2(10) — Primary Key
AUTHOR_NAME	VARCHAR2(100) NOT NULL
NATIONALITY	VARCHAR2(50)

BOOK_AUTHOR (Junction / Associative Table)	
BOOK_ID PK FK → BOOK	VARCHAR2(10)
AUTHOR_ID PK FK → AUTHOR	VARCHAR2(10)

"Satu buku boleh ada banyak pengarang — contoh buku co-authored — dan satu pengarang boleh menulis banyak buku. Ini hubungan **Many-to-Many (M:N)**. Masalahnya, relational database tidak boleh terus melukis M:N antara dua jadual; ia mesti diselesaikan dengan **junction table** (atau *associative table*) — dalam kes ini `BOOK_AUTHOR`, yang primary key-nya gabungan (composite key) daripada dua foreign key tersebut."

Soalan dijangka: "Macam mana ni berkaitan dengan 3NF?"

"Kalau AUTHOR_NAME disimpan terus dalam jadual BOOK, ia akan jadi redundant — nama pengarang akan berulang setiap kali buku yang sama muncul. Dengan memisahkan AUTHOR dan menghubungkannya melalui BOOK_AUTHOR, setiap fakta hanya disimpan SEKALI — ini contoh konkrit bagaimana schema saya mengelakkan transitive dependency."

6. BORROWING — "Jantung" Sistem (Associative Entity)

Entiti transaksi yang menghubungkan tiga entiti sekali gus.

BORROWING	
BORROW_ID	VARCHAR2(10) — Primary Key
MEMBER_ID FK → MEMBER	VARCHAR2(10) NOT NULL
BOOK_ID FK → BOOK	VARCHAR2(10) NOT NULL
LIBRARIAN_ID FK → LIBRARIAN	VARCHAR2(10) NOT NULL
BORROW_DATE	DATE NOT NULL
DUE_DATE	DATE NOT NULL — CHECK ≥ BORROW_DATE
RETURN_DATE	DATE, nullable — CHECK ≥ BORROW_DATE bila diisi
FINE_AMOUNT	NUMBER, DEFAULT 0 — CHECK ≥ 0
STATUS	VARCHAR2(10) — CHECK IN ('BORROWED','RETURNED','OVERDUE')

"Borrowing adalah 'jantung' sistem ini — ia menghubungkan **tiga** entiti sekali gus: Member (siapa meminjam), Book (apa yang dipinjam), dan Librarian (siapa yang memproses transaksi). Setiap hubungan ini **1:M** — satu Member boleh ada banyak rekod pinjaman sepanjang masa, satu Book boleh dipinjam berkali-kali (pada tarikh berlainan), dan satu Librarian boleh memproses banyak transaksi."

Tip: Sebut juga bahawa RETURN_DATE dan FINE_AMOUNT sengaja dibuat nullable/default 0 — kerana pada masa rekod dicipta (buku baru dipinjam), buku belum lagi dipulangkan. Ini menunjukkan anda faham *lifecycle* data, bukan sekadar struktur statik.

7. Business Rules → Constraints (Hubungkan ERD dengan Implementation)

Bahagian ini menunjukkan business rules bukan sekadar tulisan di atas kertas — ia dikuatkuasakan oleh database sendiri.

"Setiap business rule dalam proposal saya diterjemahkan terus jadi **CHECK constraint** pada level database — ini bermakna data yang melanggar peraturan **TIDAK BOLEH** disimpan, walaupun ada bug pada aplikasi di atasnya."

Business Rule	Constraint Database	Jadual
Baki kopi tak boleh lebih daripada jumlah kopi	<code>available_copies <= total_copies</code>	BOOK
Tarikh perlu pulang mesti selepas/sama tarikh pinjam	<code>due_date >= borrow_date</code>	BORROWING
Fine tidak boleh negatif	<code>fine_amount >= 0</code>	BORROWING
Setiap orang hanya satu role (Member ATAU Librarian)	UNIQUE pada PERSON_ID	MEMBER, LIBRARIAN
ISBN, Email, Staff ID tidak boleh berulang	UNIQUE constraint	BOOK, PERSON, LIBRARIAN
Status mesti salah satu nilai sah	<code>CHECK IN (...)</code>	BORROWING, MEMBER, PERSON

Tapi CHECK constraint ada limitasi: ia cuma boleh validate nilai dalam SATU baris/jadual pada satu masa — ia tak boleh "kira balik baki kopi" atau "kemaskini jadual lain secara automatik". Untuk business rule yang lebih kompleks seperti itu, saya guna **PL/SQL** — diterangkan dalam bahagian seterusnya.

8. PL/SQL — Function, Trigger & View

Bahagian ini membezakan "Database Design" (ERD & normalization) dengan "Database Engineering" — menunjukkan business logic dijalankan SECARA AKTIF oleh database, bukan sekadar pasif menyimpan data.

"Selain CHECK constraint yang pasif, saya implementasikan **satu function, dua trigger, dan dua view** untuk business logic yang perlu mengira atau mengemaskini jadual lain secara automatik."

FUNCTION — fn_calculate_fine

Mengira fine RM1 setiap hari lewat. Dipanggil oleh laman Return DAN proses "sweep" overdue automatik — supaya formula fine hanya wujud di SATU tempat.

```
CREATE OR REPLACE FUNCTION fn_calculate_fine (  
    p_due_date IN DATE, p_return_date IN DATE  
) RETURN NUMBER  
IS  
BEGIN  
    RETURN GREATEST(0, TRUNC(p_return_date) - TRUNC(p_due_date));  
END;
```

TRIGGER — trg_borrowing_after_insert

Setiap kali rekod BORROWING baharu dicipta, trigger ini automatik mengurangkan AVAILABLE_COPIES buku tersebut sebanyak 1 — dan **menolak insert itu** (raise error) jika baki kopi sudah 0.

```
CREATE OR REPLACE TRIGGER trg_borrowing_after_insert  
AFTER INSERT ON BORROWING  
FOR EACH ROW  
DECLARE v_rows NUMBER;  
BEGIN  
    UPDATE BOOK SET AVAILABLE_COPIES = AVAILABLE_COPIES - 1  
    WHERE BOOK_ID = :NEW.BOOK_ID AND AVAILABLE_COPIES > 0;  
    v_rows := SQL%ROWCOUNT;  
    IF v_rows = 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Book not available for borrowing');  
    END IF;  
END;
```

TRIGGER — trg_borrowing_after_return

Apabila RETURN_DATE bertukar daripada NULL kepada ada nilai (buku dipulangkan), trigger ini automatik menambah semula AVAILABLE_COPIES.

```
CREATE OR REPLACE TRIGGER trg_borrowing_after_return  
AFTER UPDATE OF RETURN_DATE ON BORROWING  
FOR EACH ROW  
WHEN (NEW.RETURN_DATE IS NOT NULL AND OLD.RETURN_DATE IS NULL)
```

```
BEGIN
  UPDATE BOOK SET AVAILABLE_COPIES = LEAST(TOTAL_COPIES, AVAILABLE_COPIES + 1)
  WHERE BOOK_ID = :NEW.BOOK_ID;
END;
```

VIEW — vw_overdue_report & vw_book_popularity

`vw_overdue_report` menyenaraikan semua pinjaman lewat (ahli, buku, hari lewat, fine) dalam satu query sedia-pakai.

`vw_book_popularity` menyusun buku mengikut kekerapan dipinjam — laporan ringkas untuk pihak pengurusan.

```
CREATE OR REPLACE VIEW vw_book_popularity AS
SELECT bk.BOOK_ID, bk.TITLE, COUNT(br.BORROW_ID) AS TIMES_BORROWED
FROM BOOK bk LEFT JOIN BORROWING br ON bk.BOOK_ID = br.BOOK_ID
GROUP BY bk.BOOK_ID, bk.TITLE
ORDER BY TIMES_BORROWED DESC;
```

Kenapa pindah logic ni dari aplikasi ke database? Sebelum ini, decrement/restock AVAILABLE_COPIES dan pengiraan fine dijalankan dalam kod TypeScript. Sekarang ia dijalankan oleh trigger/function dalam Oracle — bermakna peraturan ini **terpakai walaupun seseorang masuk terus ke database melalui SQL*Plus/SQL Developer**, tanpa melalui aplikasi web langsung. Ini contoh sebenar prinsip "*enforce business rules at the data tier, not just the app tier*".

9. Soalan Lazim Penilai & Cara Jawab

Sediakan jawapan untuk soalan-soalan yang biasa ditanya semasa Q&A.

Q: Kenapa guna VARCHAR2 untuk Primary Key, bukan NUMBER auto-increment?

A: "Supaya ID boleh diformat bermakna dan mudah dibaca manusia — contoh 'BOOK01', 'CAT01' — berbanding nombor siri semata-mata. Logic penjana ID dikendalikan di aplikasi (backend), dengan mekanisme retry jika berlaku pelanggaran (collision)."

Q: Adakah schema ini sudah berada dalam 3NF (Third Normal Form)?

A: "Ya. Tiada partial dependency (semua jadual guna single-column primary key kecuali BOOK_AUTHOR yang composite key memang sesuai untuk junction table), dan tiada transitive dependency — contohnya AUTHOR_NAME tidak disimpan terus dalam BOOK, sebaliknya dirujuk melalui BOOK_AUTHOR ke jadual AUTHOR."

Q: Kenapa PASSWORD_HASH ada dalam MEMBER dan LIBRARIAN, bukan dalam PERSON?

A: "Sebab login credential adalah spesifik kepada cara seseorang log masuk sebagai Member atau Librarian — ia bukan attribute semula jadi seorang 'Person', sebaliknya attribute akaun sistem untuk role tertentu."

Q: Kalau Member dipadam, apa jadi kepada Borrowing dia?

A: "Pada reka bentuk semasa, Borrowing merujuk kepada MEMBER_ID melalui foreign key — jadi sistem akan menghalang pemadaman jika masih ada rekod pinjaman berkaitan (referential integrity), melainkan dipadam mengikut urutan yang betul."

Q: Kenapa decrement AVAILABLE_COPIES dibuat dalam trigger, bukan terus dalam kod aplikasi?

A: "Supaya peraturan 'tak boleh pinjam kalau baki kopi 0' terpakai pada SEMUA cara data dimasukkan — termasuk insert terus melalui SQL Developer, bukan hanya melalui aplikasi web. Ini juga mengelakkan race condition diselesaikan dua kali (di app dan di DB) yang boleh bercanggah."

Q: Apa beza FUNCTION dengan PROCEDURE dalam PL/SQL?

A: "Function mesti return satu nilai dan boleh dipanggil terus dalam statement SQL (contoh dalam UPDATE...SET FINE_AMOUNT = fn_calculate_fine(...)). Procedure tidak semestinya return nilai dan dipanggil secara berasingan (contoh dalam blok BEGIN...END atau trigger). Saya pilih FUNCTION di sini sebab saya perlukan nilai fine terus di dalam SQL UPDATE statement."

10. Tips Penyampaian (Delivery)

Cara menyampaikan supaya kelihatan yakin dan faham, bukan menghafal.

1. **Tunjuk, jangan baca.** Bawa pointer/cursor terus ke kotak entiti dalam diagram bila menerangkannya — jangan baca slide tanpa merujuk visual.
2. **Sebut cardinality dengan notasi yang betul.** Dalam Crow's Foot notation: garis lurus = "one", garis bercabang tiga = "many", bulatan kecil = "optional / zero".
3. **Gunakan ayat penghubung (transition).** Contoh: "Sekarang saya akan terangkan bagaimana hubungan ini diselesaikan dalam schema sebenar..." — supaya aliran presentation tidak terputus-putus.
4. **Letak keutamaan pada LOGIK, bukan nama column.** Kalau terlupa nama column yang tepat, tak mengapa — lebih penting terangkan KENAPA relationship atau constraint itu wujud.
5. **Sediakan satu contoh data sebenar** untuk setiap relationship penting (contoh: "Buku 'Tafsir Al-Quran' ada dua pengarang, jadi ada dua baris dalam BOOK_AUTHOR") — ini buat penerangan abstrak jadi konkrit dan mudah difahami penilai.
6. **Demo PL/SQL secara live kalau boleh.** Buka SQL Developer, buat satu pinjaman buku yang baki kopinya 0 — tunjukkan ia ditolak terus oleh trigger (ORA-20001), bukan oleh kod aplikasi. Ini lebih convincing daripada cuma cerita.
7. **Akhiri dengan ringkasan.** "Jadi secara keseluruhan, reka bentuk ini mengelakkan redundancy melalui supertype/subtype PERSON, menyelesaikan Many-to-Many melalui BOOK_AUTHOR, dan menguatkuasakan semua business rule — daripada yang ringkas (CHECK constraint) hinggalah yang kompleks (trigger, function, view) — terus di level database."